

Python Programming Basics: Study Notes

Computer Science Study Notes, Part 1 of 6

1. Introduction and Environment Setup

Python is a high-level, interpreted, dynamically typed programming language created by Guido van Rossum and first released in 1991. It is known for its readable syntax: instead of curly braces, Python uses indentation to define code blocks. Python is widely used in web development, data science, automation, and artificial intelligence.

You can check which version is installed with the command `python --version`. Most modern libraries require at least Python 3.10.

A virtual environment keeps the packages of one project separate from the packages of every other project, which avoids version conflicts. To create one, run `python -m venv .venv` inside the project folder. To activate it on Windows, run `.venv\Scripts\activate`. On macOS or Linux, run `source .venv/bin/activate`.

Packages are installed with `pip`, for example `pip install pandas`. The command `pip freeze > requirements.txt` records the exact versions installed in the environment, and `pip install -r requirements.txt` installs them again on another machine. Sharing a `requirements.txt` file makes a project reproducible.

2. Data Types, Variables and Operators

The basic built-in types are `int` (whole numbers), `float` (decimal numbers), `str` (text), `bool` (True or False) and `None` (the absence of a value). Python is dynamically typed, so a variable does not need a declared type: `age = 21` creates an `int` and `name = "Sara"` creates a `str`.

Python has four main collection types:

- `list`: an ordered, mutable sequence, written with square brackets, for example `[1, 2, 3]`.
- `tuple`: an ordered, immutable sequence, written with parentheses, for example `(1, 2, 3)`.
- `dict`: a collection of key-value pairs, written with curly braces. Since Python 3.7 a dict keeps the insertion order of its keys.
- `set`: an unordered collection of unique elements. Adding a duplicate value has no effect.

Mutable objects such as lists can be changed after creation, while immutable objects such as strings and tuples cannot.

Arithmetic operators include `+`, `-`, `*`, `/`, `//`, `%` and `**`. The single slash `/` is true division and always returns a float, so `7 / 2` gives 3.5. The double slash `//` is floor division and returns the whole part, so `7 // 2` gives 3. The percent sign `%` returns the remainder, so `7 % 2` gives 1. The operator `**` raises to a power, so `2 ** 3` gives 8.

Formatted strings, called f-strings, embed expressions directly in text: `f"Hello {name}, you are {age} years old"`.

3. Control Flow and Functions

Conditional logic uses `if`, `elif` and `else`. Loops come in two forms: a `for` loop iterates over any iterable such as a list, string or range, and a `while` loop repeats as long as a condition stays true. The keyword `break` exits a loop immediately, and `continue` skips to the next iteration.

The function `range(start, stop, step)` generates numbers. The stop value is exclusive, so `range(0, 5)` produces 0, 1, 2, 3 and 4.

Functions are defined with the `def` keyword. A function can have default parameter values, and it returns a value with `return`. If no return statement runs, the function returns `None`. The special parameters `*args` and `**kwargs` collect extra positional arguments and extra keyword arguments. Type hints such as `def area(r: float) -> float:` document the expected types but are not enforced at runtime. A docstring, written in triple quotes as the first statement of a function, describes what the function does.

A list comprehension builds a list in a single expression. For example, `[x * x for x in range(5) if x % 2 == 0]` produces `[0, 4, 16]`, because only the even numbers 0, 2 and 4 pass the condition and each is squared.

4. Error Handling, Files and Modules

Errors are handled with `try` and `except`. Code that might fail goes in the `try` block, and the `except` block runs if an exception occurs. An optional `else` block runs when no exception happened, and a `finally` block always runs, which makes it suitable for cleanup. You can raise your own exception with `raise ValueError("message")`.

Common built-in exceptions include `ValueError` (right type, wrong value), `TypeError` (wrong type), `KeyError` (missing dictionary key), `FileNotFoundError` (file does not exist) and `ZeroDivisionError` (division by zero).

Files are opened with `open()`. The recommended pattern is a context manager: `with open("notes.txt", "r", encoding="utf-8") as f:`. The `with` statement closes the file automatically, even if an error occurs. Always pass `encoding="utf-8"` explicitly, because otherwise the default encoding depends on the operating system, and this often corrupts Arabic and other non-English text on Windows.

Code in one file can be reused in another with `import`. The idiom `if __name__ == "__main__":` makes a block run only when the file is executed directly, and not when it is imported as a module.